

UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO
Facultad de Ciencias



Introducción a las Ciencias de la Computación

Notas

Profesor:

Salvador López Mendoza

Ayudante:

Kevin Jair Torres Valencia

Ayudante de Laboratorio:

Miriam Torres Bucio

Índice

1. Proceso de programacion	5
1.1. Planteamiento del problema	5
1.2. Análisis del problema	6
1.3. Diseño de la solución	6
1.3.1. Metodología de diseno	6
1.4. Codificación	7
1.5. Pruebas y depuración	7
1.6. Documentación	7
1.7. Mantenimiento	8
1.8. Ejemplos	8
2. Creacion y uso de datos primitivos	13
2.1. Identificadores	13
2.2. Declaracion de datos	13
2.3. Uso de datos (operadores)	14
2.3.1. Operador de asignacion	14
2.3.2. Operadores aritmeticos	14
2.3.3. Operador de asignacion compuesto	15
2.3.4. Operadores de relacion	15
2.3.5. Operadores logicos	15
2.3.6. Operador + para cadenas	15
2.3.7. Precedencia y asociacion de operadores	15
2.4. Conversion de tipo	16
2.5. Ejemplo	16
3. Creacion y uso de objetos	17
3.1. Creacion de objetos	17
3.2. Uso de objetos	17
3.3. Eliminacion de objetos	17
3.4. Objetos para lectura y escritura	17
3.5. Ejemplos de trabajo con objetos	18
4. Creacion y uso de clases	18
4.1. Definicion de una clase	18
4.2. Atributos	19
4.3. Metodos	19
4.4. Creacion de objetos	19
4.5. Constructores	20
4.6. Encapsulamiento	20
4.7. Metodos de acceso	21

5. Objetos como atributos	21
5.1. Abstraccion en el diseño de clases	21
5.2. Objetos que crean objetos	22
5.3. Interaccion de objetos	22
5.4. Agregacion	22
6. Agrupación de objetos	23
6.1. Introduccion	23
6.2. Creacion y uso de arreglos	23
6.3. Metodos que devuelven arreglos	23
6.4. Arreglos como parametros	24
6.5. Arreglos de cadenas	24
6.5.1. Parametros del metodo main	24
6.6. Arreglos de objetos	24
6.7. Arreglos bidimensionales	25
6.8. Procesamiento de arreglos	25
6.9. Ventajas de la agrupacion	25
7. Herencia de clases	26
7.1. Ampliacion mediante herencia	26
7.2. Control de acceso	26
7.3. Constructores	27
7.4. Uso de clases derivadas	27
7.5. Especializacion mediante herencia	27
7.6. Jerarquia de clases	28
7.7. Ventajas de la herencia	28
7.8. Compatibilidad	28
7.9. Generalizacion mediante herencia	29
7.10. La clase Object	29
7.11. Sobreescritura de metodos	29
7.12. Polimorfismo	29
8. Clase Exception	30
8.1. Excepciones	30
8.2. Jerarquía de excepciones	30
8.3. Estados de una excepción	31
8.3.1. Disparo de excepciones	31
8.3.2. Manejo de excepciones	31
8.3.3. Bloque try	32
8.3.4. Bloque catch	32
8.3.5. Bloque finally	32
8.4. Lanzamiento de excepciones	32
8.5. Propagación de excepciones	33
8.6. La clausula throws	33

8.7. Excepciones verificadas y no verificadas	33
8.7.1. Excepciones verificadas	33
8.7.2. Excepciones no verificadas	33
8.8. Creación de excepciones propias	34
8.9. Recuperación de excepciones	34
8.10. Ventajas	34
9. Clases abstractas e interfaces	35
9.1. Clases abstractas	35
9.1.1. Métodos abstractos	35
9.2. Jerarquía de clases abstractas	36
9.3. Interfaces	36
9.3.1. Implementación de interfaces	36
9.4. Referencias a interfaces	36
9.5. Implementación de más de una interfaz	37
9.6. Definición de grupos de constantes	37
9.7. Interfaces definidas en Java	37
9.8. Clases abstractas vs. interfaces	37
10. Serialización de objetos	38
10.1. Persistencia de objetos	38
10.2. Interfaz Serializable	39
10.3. Escritura de objetos	39
10.3.1. Flujo de salida	39
10.3.2. Escritura de objetos serializados	39
10.4. Lectura de objetos	39
10.4.1. Flujo de entrada	40
10.4.2. Lectura de objetos serializados	40
10.5. Conversión de objetos	40
10.6. Objetos no serializables	40
10.6.1. Atributos transient	40
10.7. Versión de clases	41
10.8. Excepciones en serialización	41
10.9. Ventajas de la serialización	41
Apéndice A. Arreglos	42

1. Proceso de programacion

El proceso de programación consiste en un conjunto de actividades necesarias para desarrollar programas que resuelvan correctamente un problema específico. Estas actividades incluyen: definición del problema, diseño de la solución, implementación, depuración, pruebas, documentación y mantenimiento.

El proceso no es lineal sino **iterativo e incremental**. Durante el desarrollo es común regresar a etapas anteriores para mejorar o corregir la solución.

Programar implica más que escribir código. Antes de implementar una solución es necesario comprender el problema y diseñar una estrategia para resolverlo.

Las principales etapas del proceso de programación son:

- **Planteamiento del problema.**
- **Análisis del problema.**
- **Diseño de la solución.**
- **Codificación.**
- **Pruebas y depuración.**
- **Documentación.**
- **Mantenimiento.**

La documentación se realiza durante todo el proceso para registrar cómo y por qué se construyó la solución.

1.1. Planteamiento del problema

En esta etapa se determina con precisión **qué debe hacer el programa**. Una definición incorrecta o incompleta del problema conduce frecuentemente a soluciones inadecuadas.

Para definir el problema se deben identificar:

- El objetivo del programa.
- Los requerimientos del usuario.
- Las características que debe cumplir la solución.

Los requerimientos iniciales suelen ser ambiguos o incompletos, por lo que el programador debe analizarlos y refinarlos hasta eliminar ambigüedades.

1.2. Análisis del problema

Consiste en descomponer el problema en partes más pequeñas y manejables. Se analizan las relaciones entre los datos de entrada, los procesos necesarios y los resultados que se deben obtener.

En esta etapa **no se programa**, únicamente se razona la solución.

1.3. Diseño de la solución

El diseño consiste en definir una estrategia para resolver el problema mediante un programa.

Un buen diseño permite:

- organizar la solución del problema,
- identificar los componentes principales,
- establecer relaciones entre ellos,
- facilitar modificaciones futuras.

Existen diferentes formas de diseñar una solución.

1.3.1. Metodología de diseño

Una forma tradicional de diseño es describir paso a paso el procedimiento para resolver el problema.

Este procedimiento se conoce como **algoritmo**.

Un algoritmo es una descripción ordenada de los pasos necesarios para resolver un problema, con instrucciones claras y sin ambigüedades, que produce siempre el mismo resultado si se ejecuta bajo las mismas condiciones.

Los algoritmos pueden expresarse mediante:

- pseudocódigo,
- diagramas,
- o directamente mediante un programa.

En la programación orientada a objetos el diseño también implica identificar **objetos**, sus responsabilidades y la forma en que interactúan mediante mensajes.

1.4. Codificación

Consiste en traducir el diseño a un lenguaje de programación, por ejemplo Java. En esta etapa se escribe el código fuente del programa siguiendo el algoritmo o modelo diseñado previamente.

Se debe cuidar el uso correcto de:

- La sintaxis del lenguaje
- Los tipos de datos
- Las estructuras de control

Posteriormente el programa se **compila** y se **ejecuta** para verificar su funcionamiento.

1.5. Pruebas y depuración

La depuración es el proceso de localizar y corregir errores en un programa. Los errores pueden ser de distintos tipos:

- errores de sintaxis,
- errores de ejecución,
- errores lógicos.

Durante la depuración se revisa el código y se realizan pruebas para asegurar que el programa funcione correctamente.

1.6. Documentación

La documentación consiste en registrar información sobre el programa, incluyendo:

- descripción del problema,
- diseño de la solución,
- funcionamiento del programa,
- instrucciones para su uso.

Una buena documentación facilita el mantenimiento y la comprensión del sistema por parte de otros programadores.

1.7. Mantenimiento

El mantenimiento consiste en realizar modificaciones al programa después de su desarrollo inicial. Estas modificaciones pueden realizarse para:

- agregar nuevas funcionalidades,
- mejorar el desempeño,
- corregir errores detectados posteriormente.

El mantenimiento es una etapa importante porque la mayoría de los programas evolucionan con el tiempo.

1.8. Ejemplos

Ejemplo 1: Cálculo del promedio de tres calificaciones

1. Planteamiento del problema

Desarrollar un programa que solicite tres calificaciones de un alumno y calcule su promedio. El programa debe indicar si el alumno está aprobado (promedio mayor o igual a 6) o reprobado.

2. Análisis del problema

Entradas:

- Calificación 1
- Calificación 2
- Calificación 3

Proceso:

- Sumar las tres calificaciones
- Dividir entre 3
- Evaluar si el promedio es mayor o igual a 6

Salida:

- Promedio
- Mensaje de aprobado o reprobado

3. Diseño de la solución (Algoritmo en pseudocódigo)

Inicio

Leer cal1, cal2, cal3

promedio = (cal1 + cal2 + cal3) / 3

Mostrar promedio

Si promedio >= 6 entonces

Mostrar "Aprobado"

Sino

Mostrar "Reprobado"

Fin

4. Codificación en Java

```
import java.util.Scanner;
```

```
public class PromedioAlumno {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double cal1, cal2, cal3, promedio;

        System.out.print("Ingrese la primera calificación: ");
        cal1 = sc.nextDouble();

        System.out.print("Ingrese la segunda calificación: ");
        cal2 = sc.nextDouble();

        System.out.print("Ingrese la tercera calificación: ");
        cal3 = sc.nextDouble();

        promedio = (cal1 + cal2 + cal3) / 3;

        System.out.println("Promedio: " + promedio);

        if (promedio >= 6) {
            System.out.println("Aprobado");
        } else {
            System.out.println("Reprobado");
        }

        sc.close();
    }
}
```

5. Pruebas

Entrada: 8, 7, 9

Salida esperada: Promedio = 8.0 → Aprobado

Entrada: 5, 6, 4

Salida esperada: Promedio = 5.0 → Reprobado

6. Documentación

El programa calcula el promedio de tres calificaciones y determina la situación académica del alumno.

7. Mantenimiento

Podría modificarse para permitir cualquier número de calificaciones usando ciclos.

Ejemplo 2: Determinar si un número es par o impar

1. Planteamiento del problema

Desarrollar un programa que determine si un número entero ingresado por el usuario es par o impar.

2. Análisis del problema

Entrada:

- Número entero

Proceso:

- Obtener el residuo de dividir el número entre 2
- Si el residuo es 0, el número es par
- En otro caso, es impar

Salida:

- Mensaje indicando si es par o impar

3. Diseño de la solución (Algoritmo)

Inicio

Leer numero

Si numero mod 2 = 0 entonces

Mostrar "Es par"

Sino

Mostrar "Es impar"

Fin

4. Codificación en Java

```
import java.util.Scanner;

public class NumeroParImpar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int numero;

        System.out.print("Ingrese un número entero: ");
        numero = sc.nextInt();

        if (numero % 2 == 0) {
            System.out.println("El número es par");
        } else {
            System.out.println("El número es impar");
        }

        sc.close();
    }
}
```

5. Pruebas

Entrada: 10 → Salida: Es par

Entrada: 7 → Salida: Es impar

6. Documentación

El programa determina la paridad de un número utilizando el operador módulo.

7. Mantenimiento

Podría ampliarse para validar que el usuario no ingrese caracteres no numéricos.

Ejemplo 3: Conversión de grados Celsius a Fahrenheit

1. Planteamiento del problema

Desarrollar un programa que convierta una temperatura dada en grados Celsius a grados Fahrenheit.

2. Análisis del problema

Entrada:

- Temperatura en grados Celsius

Proceso: La fórmula de conversión es:

$$F = \frac{9}{5}C + 32$$

Salida:

- Temperatura en grados Fahrenheit

3. Diseño de la solución (Algoritmo)

Inicio

Leer C

$F = (9/5) * C + 32$

Mostrar F

Fin

4. Codificación en Java

```
import java.util.Scanner;
```

```
public class ConversionTemperatura {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        double celsius, fahrenheit;  
  
        System.out.print("Ingrese la temperatura en Celsius: ");  
        celsius = sc.nextDouble();  
  
        fahrenheit = (9.0 / 5.0) * celsius + 32;  
  
        System.out.println("Temperatura en Fahrenheit: " + fahrenheit);  
  
        sc.close();  
    }  
}
```

5. Pruebas

Entrada: 0

Salida esperada: 32

6. Documentación

El programa realiza una conversión directa usando una fórmula matemática sin estructuras de control.

7. Mantenimiento

Podría ampliarse para convertir también de Fahrenheit a Celsius.

2. Creacion y uso de datos primitivos

Los **datos primitivos** son tipos de datos predefinidos por el lenguaje Java que permiten representar valores simples como números o valores lógicos.

Estos tipos se utilizan para almacenar información básica y realizar operaciones mediante expresiones.

2.1. Identificadores

Un identificador es el nombre que se asigna a variables, métodos o clases. Para definir identificadores se deben seguir ciertas reglas:

- Deben comenzar con una letra, guion bajo.
- No pueden comenzar con un número.
- No pueden coincidir con palabras reservadas del lenguaje.

Los identificadores deben elegirse de forma que describan claramente su propósito. Existe un conjunto de palabras que no pueden ser usadas como identificadores porque tiene un significado especial para java, cada una de éstas se denomina palabra reservada.

2.2. Declaracion de datos

La declaración de una variable consiste en especificar el tipo de dato que almacenará y el identificador que la representará.

La forma general es:

```
tipo identificador;
```

También es posible inicializar la variable en el momento de declararla:

```
tipo identificador = valor;
```

Los tipos de datos definidos en java, y denominados primitivos, son:

- Tipo numérico (enteros o reales).
- Tipo carácter.
- Tipo Booleano.

Tipo	Descripción
byte	Entero de 8 bits
short	Entero de 16 bits
int	Entero de 32 bits
long	Entero de 64 bits
float	Real en 32 bits con 7 dígitos en la parte decimal
double	Real en 64 bits con 15 dígitos en la parte decimal
char	Carácter en 16-bits (Unicode)
boolean	Booleano

2.3. Uso de datos (operadores)

Los datos primitivos se utilizan principalmente en expresiones que emplean operadores.

2.3.1. Operador de asignación

El operador de asignación (=) se utiliza para almacenar un valor en una variable.

```
x = 5;
```

Esto indica que el valor 5 se asigna a la variable `x`.

2.3.2. Operadores aritméticos

Los operadores aritméticos permiten realizar operaciones matemáticas.

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
%	Residuo de la división

2.3.3. Operador de asignacion compuesto

Los operadores de asignación compuesta combinan una operación aritmética con una asignación.

Ejemplos:

```
x += 5;  
x -= 3;  
x *= 2;  
x /= 4;
```

2.3.4. Operadores de relacion

Los operadores de relación permiten comparar valores y producen un resultado booleano.

- ==: Asignacion.
- !=: Negacion.
- <: Menor.
- >: Mayor.
- <=: Menor igual que.
- >=: Mayor igual que.

2.3.5. Operadores logicos

Los operadores lógicos se utilizan para combinar expresiones booleanas.

- AND lógico: &&
- OR lógico: ||
- NOT lógico: !

2.3.6. Operador + para cadenas

El operador + también puede utilizarse para concatenar cadenas de texto.

```
String mensaje = "Hola " + nombre;
```

2.3.7. Precedencia y asociacion de operadores

Cuando una expresión contiene varios operadores, Java evalúa la expresión siguiendo reglas de **precedencia** y **asociación**.

Estas reglas determinan el orden en que se ejecutan las operaciones.

2.4. Conversion de tipo

La conversión de tipo permite transformar un valor de un tipo de dato a otro.

Puede ser:

- **implícita:** realizada automáticamente por el compilador.
- **explícita:** indicada por el programador mediante casting.

Ejemplo de conversión implícita:

```
int x = 42;
double numeroDecimal = x;
```

Ejemplo de conversión explícita:

```
int x = (int) 3.5;
```

2.5. Ejemplo

```
public class DatosPrimitivos {

    public static void main(String[] args) {

        int edad = 21;
        double estatura = 1.75;
        char grupo = 'A';
        boolean inscrito = true;

        System.out.println("Edad: " + edad);
        System.out.println("Estatura: " + estatura);
        System.out.println("Grupo: " + grupo);
        System.out.println("Inscrito: " + inscrito);

        double edadFutura = edad + 5;

        System.out.println("Edad en 5 años: " + edadFutura);
    }
}
```


3. Creacion y uso de objetos

En la programación orientada a objetos, los programas se construyen mediante la interacción de objetos.

Un **objeto** representa una entidad del problema que se desea modelar y posee:

- **estructura**: atributos o datos que describen al objeto,
- **comportamiento**: operaciones que el objeto puede realizar,
- **identidad**: característica que distingue a un objeto de otro.

3.1. Creacion de objetos

Los objetos se crean a partir de una clase utilizando el operador **new**.

Ejemplo:

```
Scanner entrada = new Scanner(System.in);
```

La variable almacena una **referencia** al objeto creado.

3.2. Uso de objetos

Los objetos interactúan entre sí mediante el envío de **mensajes**. En Java, un mensaje corresponde a la invocación de un método.

Ejemplo:

```
entrada.nextInt();
```

El método se ejecuta sobre el objeto indicado por la referencia.

3.3. Eliminacion de objetos

Cuando un objeto deja de ser utilizado y no existen referencias hacia él, el sistema puede liberar la memoria que ocupa.

Este proceso es gestionado automáticamente por el **recolector de basura** (garbage collector).

3.4. Objetos para lectura y escritura

Java proporciona clases para facilitar la interacción con el usuario. Una de las más utilizadas es la clase **Scanner**, que permite leer datos desde la entrada estándar.

Ejemplo:

```
Scanner sc = new Scanner(System.in);  
int numero = sc.nextInt();
```

3.5. Ejemplos de trabajo con objetos

El trabajo con objetos incluye operaciones como:

- creación de objetos,
- envío de mensajes (llamada a métodos),
- recepción de valores devueltos por métodos,
- interacción entre varios objetos.

La colaboración entre objetos permite dividir un problema complejo en componentes más simples.

4. Creacion y uso de clases

En la programación orientada a objetos, una **clase** es un modelo o plantilla que define las características y el comportamiento de los objetos.

Una clase describe:

- **Atributos:** datos que describen el estado del objeto.
- **Métodos:** operaciones que el objeto puede realizar.

Los objetos son **instancias** de una clase y representan entidades concretas dentro de un programa.

4.1. Definicion de una clase

Una clase se define utilizando la palabra reservada **class** seguida del nombre de la clase.

La estructura general es:

```
class NombreClase {  
  
    // atributos  
    tipo atributo;  
  
    // metodos  
    tipo metodo() {  
        // instrucciones  
    }  
}
```

Por convención en Java, los nombres de las clases comienzan con letra mayúscula.

4.2. Atributos

Los atributos representan las características o propiedades de los objetos de una clase.

Cada objeto tiene su propio conjunto de valores para estos atributos.

Ejemplo:

```
class Persona {  
    String nombre;  
    int edad;  
}
```

En este ejemplo, cada objeto de tipo `Persona` tendrá un nombre y una edad.

4.3. Metodos

Los métodos definen el comportamiento de los objetos. Un método contiene instrucciones que permiten realizar operaciones sobre los datos del objeto.

Ejemplo:

```
class Persona {  
  
    String nombre;  
    int edad;  
  
    void saludar() {  
        System.out.println("Hola");  
    }  
}
```

Los métodos pueden:

- recibir parámetros,
- devolver valores,
- modificar los atributos del objeto.

4.4. Creacion de objetos

Los objetos se crean a partir de una clase utilizando el operador `new`.

Ejemplo:

```
Persona p = new Persona();
```

La variable `p` almacena una referencia al objeto creado. Una vez creado el objeto, se pueden acceder a sus atributos y métodos mediante el operador punto.

```
p.nombre = "Ana";  
p.saludar();
```

4.5. Constructores

Un constructor es un método especial que se utiliza para inicializar los objetos cuando son creados. El constructor tiene el mismo nombre que la clase y no tiene tipo de retorno.

Ejemplo:

```
class Persona {  
  
    String nombre;  
    int edad;  
  
    Persona(String n, int e) {  
        nombre = n;  
        edad = e;  
    }  
}
```

La creación del objeto se realiza de la siguiente forma:

```
Persona p = new Persona("Ana", 20);
```

4.6. Encapsulamiento

El encapsulamiento consiste en ocultar los detalles internos de una clase y controlar el acceso a sus atributos. Para lograrlo se utilizan modificadores de acceso como:

- `private`
- `public`
- `protected`

Generalmente los atributos se declaran como `private` y se accede a ellos mediante métodos.

4.7. Metodos de acceso

Los métodos de acceso permiten leer o modificar los valores de los atributos. Estos métodos se conocen comúnmente como:

- **getters (obtener)**: devuelven el valor de un atributo.
- **setters (asignar)**: modifican el valor de un atributo.

Ejemplo:

```
class Persona {  
  
    private String nombre;  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String n) {  
        nombre = n;  
    }  
}
```

Este mecanismo permite controlar cómo se utilizan los datos de un objeto.

5. Objetos como atributos

En programación orientada a objetos, un objeto puede formar parte de otro objeto como uno de sus atributos. Esta relación permite modelar estructuras más complejas y representar entidades reales mediante la composición de objetos.

Cuando un objeto contiene a otros como atributos, se establece una relación de **agregación**, en la cual un objeto se construye utilizando otros objetos como componentes.

5.1. Abstraccion en el diseño de clases

La abstracción consiste en identificar únicamente las características relevantes de un objeto para resolver un problema.

Permite ignorar detalles innecesarios y concentrarse en los elementos esenciales.

- Facilita el diseño de programas grandes.
- Permite modelar objetos del mundo real.
- Ayuda a dividir problemas complejos.

Una misma entidad puede abstraerse de distintas formas dependiendo del problema que se esté resolviendo.

5.2. Objetos que crean objetos

Un objeto puede encargarse de crear otros objetos durante la ejecución del programa. Esto ocurre cuando dentro de un método se utiliza el operador **new** para generar instancias adicionales.

Este mecanismo permite:

- delegar responsabilidades,
- modularizar tareas,
- reutilizar componentes.

5.3. Interaccion de objetos

Los objetos colaboran entre sí mediante el intercambio de mensajes.

La solución de un problema generalmente requiere la cooperación de varios objetos, donde cada uno realiza parte del trabajo.

La interacción entre objetos se basa en:

- envío de mensajes,
- ejecución de métodos,
- modificación de estados.

La colaboración permite distribuir responsabilidades y construir soluciones más organizadas.

5.4. Agregacion

La agregación es una relación en la cual un objeto contiene otros objetos como parte de su estructura.

Características:

- representa una relación “tiene un”,
- permite construir objetos complejos,
- favorece la reutilización.

Ejemplo:

- una línea tiene dos puntos,
- un automóvil tiene un motor,
- una cuenta tiene un cliente.

6. Agrupación de objetos

La agrupación de objetos permite almacenar varios elementos del mismo tipo en una sola estructura. En Java, esta agrupación se realiza mediante arreglos.

Un arreglo permite trabajar con múltiples datos relacionados de forma organizada.

6.1. Introduccion

Cuando se requiere manejar varios datos del mismo tipo, resulta conveniente agruparlos en una sola estructura.

El uso de arreglos evita declarar múltiples variables individuales para almacenar información similar.

Ventajas:

- organización,
- acceso por índice,
- procesamiento repetitivo.

6.2. Creacion y uso de arreglos

Un arreglo es un objeto que almacena un conjunto fijo de elementos del mismo tipo.

Se crea indicando:

- tipo de datos,
- tamaño.

Ejemplo:

```
int[] numeros = new int[10];
```

Cada posición del arreglo se identifica mediante un índice, comenzando en cero.

6.3. Metodos que devuelven arreglos

Un método puede devolver un arreglo como resultado.

Esto permite generar colecciones de datos y reutilizarlas en otras partes del programa.

Ejemplo conceptual:

- crear arreglo,
- llenarlo,
- devolverlo.

Es útil cuando un método produce varios resultados relacionados.

6.4. Arreglos como parametros

Un arreglo puede enviarse como argumento a un método. Cuando esto sucede, el método recibe la referencia al arreglo y puede trabajar directamente sobre sus elementos.

Esto permite:

- modificar el arreglo,
- recorrerlo,
- procesar datos agrupados.

6.5. Arreglos de cadenas

Los arreglos también pueden almacenar objetos de tipo `String`. Se utilizan para guardar textos relacionados.

Ejemplo:

```
String[] nombres = new String[5];
```

6.5.1. Parametros del metodo main

El método `main` recibe un arreglo de cadenas:

```
String[] args
```

Este arreglo almacena los argumentos proporcionados al ejecutar el programa. Cada argumento se almacena como una cadena.

6.6. Arreglos de objetos

Un arreglo también puede almacenar objetos. En este caso cada posición guarda una referencia a un objeto.

Ejemplo:

```
Alumno[] grupo = new Alumno[20];
```

Cada elemento puede contener un objeto diferente de la misma clase. Esto permite modelar colecciones de entidades.

6.7. Arreglos bidimensionales

Un arreglo bidimensional es una estructura de arreglos. Se utiliza para representar datos organizados en filas y columnas.

Ejemplo:

```
int[] [] matriz = new int[3][4];
```

Aplicaciones:

- tablas,
- matrices,
- tableros,
- registros.

Cada elemento se accede mediante dos índices.

6.8. Procesamiento de arreglos

Las operaciones más comunes sobre arreglos son:

- recorrer,
- buscar,
- modificar,
- ordenar.

Generalmente se utilizan ciclos para trabajar con todos los elementos del arreglo.

6.9. Ventajas de la agrupación

La agrupación mediante arreglos permite:

- simplificar programas,
- reducir variables,
- representar colecciones,
- facilitar algoritmos de procesamiento.

Es una herramienta fundamental para estructuras de datos más complejas.

7. Herencia de clases

La herencia es un mecanismo de la programación orientada a objetos que permite definir nuevas clases a partir de otras clases existentes.

La nueva clase hereda atributos y métodos de la clase original, lo que favorece la reutilización de código y la organización jerárquica de clases.

La clase existente se denomina **superclase** y la nueva clase se denomina **subclase**.

La herencia modela relaciones del tipo:

- “es un”.

Ejemplo:

- Un alumno es una persona.
- Un automóvil es un vehículo.

7.1. Ampliación mediante herencia

La herencia permite extender una clase ya existente.

Una subclase puede:

- reutilizar atributos,
- reutilizar métodos,
- agregar nuevos atributos,
- agregar nuevos métodos.

Esto permite construir nuevas clases sin redefinir completamente el comportamiento.

7.2. Control de acceso

Los atributos y métodos heredados conservan sus niveles de acceso.

Los modificadores de acceso principales son:

- `private`
- `protected`
- `public`

- **private:** solo accesible dentro de la misma clase.
- **protected:** accesible en subclases.
- **public:** accesible desde cualquier clase.

El control de acceso regula qué elementos pueden heredarse y utilizarse.

7.3. Constructores

Los constructores no se heredan directamente.

Sin embargo, una subclase puede invocar el constructor de su superclase mediante la palabra reservada **super**.

Esto permite inicializar correctamente la parte heredada del objeto.

Ejemplo conceptual:

```
super(parametros);
```

7.4. Uso de clases derivadas

Una vez creada una subclase, sus objetos pueden utilizar tanto los elementos heredados como los nuevos elementos definidos.

Una subclase combina:

- funcionalidad heredada,
- funcionalidad adicional.

Esto permite construir soluciones más específicas.

7.5. Especialización mediante herencia

La herencia también permite especializar una clase. Una subclase puede representar una versión más específica de una clase general.

Ejemplo:

- Cuenta
- CuentaDeAhorro
- CuentaDeCredito

Cada subclase hereda la estructura general y añade características particulares.

7.6. Jerarquía de clases

Las relaciones de herencia forman una jerarquía. Una jerarquía organiza clases de lo general a lo particular.

Características:

- la superclase está en niveles superiores,
- las subclases derivan de ella,
- puede existir varios niveles.

Esto facilita el diseño de programas extensibles.

7.7. Ventajas de la herencia

La herencia proporciona varias ventajas:

- reutilización de código,
- reducción de duplicidad,
- facilidad de mantenimiento,
- organización jerárquica,
- especialización.

También favorece la claridad del diseño orientado a objetos.

7.8. Compatibilidad

Un objeto de una subclase puede utilizarse como si fuera de su superclase. Esto se conoce como compatibilidad de tipos.

Permite:

- asignar una subclase a una referencia de superclase,
- trabajar de manera general con diferentes objetos.

Ejemplo conceptual:

```
Persona p = new Alumno();
```

7.9. Generalización mediante herencia

La generalización consiste en identificar características comunes y colocarlas en una superclase. Esto evita repetir atributos y métodos en varias clases similares.

Beneficios:

- diseño más abstracto,
- reutilización,
- simplificación.

7.10. La clase `Object`

En Java todas las clases heredan directa o indirectamente de la clase `Object`. Por ello, todas las clases disponen de métodos básicos definidos en ella.

Entre los métodos más importantes se encuentran:

- `toString()`
- `equals()`
- `hashCode()`

La clase `Object` es la raíz de toda jerarquía en Java.

7.11. Sobreescritura de métodos

Una subclase puede redefinir un método heredado para adaptar su comportamiento. Esto se denomina **sobreescritura**.

Permite modificar el comportamiento sin cambiar la estructura heredada. Se utiliza cuando la subclase necesita una implementación específica.

7.12. Polimorfismo

El polimorfismo permite tratar objetos de distintas subclases mediante una referencia de la superclase. Esto hace posible que una misma instrucción opere con diferentes tipos de objetos.

Ventajas:

- flexibilidad,
- reutilización,

- diseño más general.

El método ejecutado depende del tipo real del objeto en tiempo de ejecución.

8. Clase `Exception`

La clase `Exception` permite manejar situaciones anómalas durante la ejecución de un programa. Su uso facilita la construcción de programas robustos, es decir, programas preparados para continuar funcionando aun cuando ocurran errores inesperados.

Una excepción es un evento que altera el flujo normal de ejecución del programa. En Java, las excepciones son objetos que contienen información sobre el error ocurrido y pueden ser manejadas por el método que las genera o por alguno de los métodos que lo invocaron.

8.1. Excepciones

Java detecta automáticamente diversos errores en tiempo de ejecución, por ejemplo:

- División entre cero.
- Acceso fuera de rango en arreglos.
- Referencias nulas.
- Apertura de archivos inexistentes.

Cuando ocurre uno de estos errores, Java indica el tipo de excepción, el método y la línea donde sucedió.

Las excepciones también pueden usarse para modelar errores previstos por el programador, como datos inválidos o estados no permitidos en la aplicación.

8.2. Jerarquía de excepciones

La clase `Exception` pertenece al paquete `java.lang` y forma parte de una jerarquía de clases.

La raíz completa es:

`Object` $\rightarrow$ `Throwable` $\rightarrow$ `Exception`

A partir de `Exception` derivan muchas clases específicas, como:

- `ArithmeticException`
- `NullPointerException`

- `IllegalArgumentException`
- `FileNotFoundException`
- `InputMismatchException`

La subclase `RuntimeException` agrupa excepciones frecuentes que ocurren durante la ejecución y no requieren ser declaradas explícitamente.

8.3. Estados de una excepción

Una excepción pasa por tres estados:

1. **Disparo:** se genera la excepción cuando ocurre el error.
2. **Atrapado:** algún manejador la detecta y procesa.
3. **Terminación:** si nadie la maneja, el programa termina abruptamente.

La excepción se propaga entre métodos hasta encontrar uno que la maneje.

8.3.1. Disparo de excepciones

Para generar una excepción se utiliza la instrucción:

`throw`

Esta instrucción interrumpe inmediatamente la ejecución del método. Si la excepción no es de tipo `RuntimeException`, debe declararse en la firma del método mediante:

`throws`

Esto avisa al usuario del método que puede ocurrir una situación anómala.

8.3.2. Manejo de excepciones

El manejo de excepciones permite controlar errores para evitar que el programa termine abruptamente. Se realiza mediante bloques especiales:

- `try`
- `catch`
- `finally`
- `try`: contiene el código que puede generar la excepción.
- `catch`: define cómo manejar cada tipo de excepción.
- `finally`: código que siempre se ejecuta.

Si ocurre una excepción, se interrumpe el bloque `try` y se ejecuta el `catch` correspondiente. Si no ocurre ninguna, los `catch` se omiten.

8.3.3. Bloque try

El bloque `try` contiene el código que puede producir una excepción.

Ejemplo:

```
try {  
    // instrucciones  
}
```

Si ocurre una excepción dentro del bloque, se transfiere el control al bloque correspondiente.

8.3.4. Bloque catch

El bloque `catch` captura y maneja la excepción.

Permite ejecutar instrucciones específicas cuando ocurre un error.

Ejemplo:

```
catch(Exception e) {  
    // manejo  
}
```

El parámetro representa el objeto excepción generado.

8.3.5. Bloque finally

El bloque `finally` contiene instrucciones que siempre se ejecutan, exista o no excepción. Se utiliza normalmente para:

- cerrar archivos,
- liberar recursos,
- finalizar procesos.

8.4. Lanzamiento de excepciones

Una excepción también puede generarse explícitamente mediante la palabra reservada:

```
throw
```

Esto permite al programador indicar que ocurrió una condición inválida.

Ejemplo conceptual:

```
throw new Exception();
```


8.5. Propagación de excepciones

Si un método genera una excepción y no la maneja, esta se propaga al método que lo invocó. El proceso continúa hasta encontrar un manejador.

La propagación se reconoce porque:

- El método declara la excepción con `throws`.
- No existe bloque `try-catch` en su interior.

Esto permite delegar el tratamiento a niveles superiores del programa.

8.6. La clausula `throws`

La palabra reservada `throws` indica que un método puede generar excepciones. Se coloca en la definición del método.

Ejemplo:

```
void metodo() throws Exception
```

Esto informa que quien invoque el método debe considerar el manejo de esa excepción.

8.7. Excepciones verificadas y no verificadas

Java clasifica las excepciones en:

- verificadas,
- no verificadas.

8.7.1. Excepciones verificadas

Son aquellas que el compilador obliga a manejar. Generalmente corresponden a errores pre-visibles. Ejemplo:

- lectura de archivos,
- entrada/salida.

8.7.2. Excepciones no verificadas

No requieren manejo obligatorio. Suelen relacionarse con errores de programación. Ejemplo:

- acceso fuera de rango,
- división entre cero,
- referencias nulas.

8.8. Creación de excepciones propias

Cuando las excepciones existentes no describen adecuadamente el problema, se pueden definir nuevas clases extendiendo `Exception`.

Esto permite:

- Identificar errores específicos.
- Mejorar la claridad del código.
- Organizar errores en jerarquías propias.

Las excepciones personalizadas no añaden comportamiento adicional; principalmente mejoran la legibilidad y organización del manejo de errores.

8.9. Recuperación de excepciones

No basta con detectar el error; también es deseable permitir que el programa continúe.

Una estrategia común es repetir la operación dentro de un ciclo hasta que la entrada sea válida o se alcance un límite de intentos. Esto hace posible:

- Corregir errores interactivos.
- Evitar terminar el programa.
- Reintentar operaciones fallidas.

La recuperación es una parte importante de la robustez del software.

8.10. Ventajas

El uso de excepciones ofrece varias ventajas:

1. Separan el código normal del manejo de errores.
2. Permiten clasificar errores por tipo.
3. Facilitan crear errores específicos del problema.
4. Mejoran la claridad y mantenimiento del programa.
5. Obligan a considerar situaciones anómalas.

El manejo adecuado de excepciones es fundamental para desarrollar software confiable.

9. Clases abstractas e interfaces

Las clases abstractas y las interfaces permiten diseñar programas más generales y flexibles. Ambas se utilizan para definir comportamientos comunes entre varias clases, favoreciendo la reutilización y el polimorfismo.

9.1. Clases abstractas

Una clase abstracta representa un concepto general del cual no es posible crear objetos directamente. Se utiliza cuando se conoce parte del comportamiento común, pero ciertos métodos dependen de las subclasses.

Una clase abstracta se declara con la palabra reservada:

```
abstract
```

Puede contener:

- Métodos abstractos.
- Métodos concretos.
- Atributos.
- Constructores.

No es posible crear objetos de una clase abstracta, ya que sus métodos abstractos no tienen implementación.

9.1.1. Métodos abstractos

Un método abstracto declara solamente la firma del método y obliga a las subclasses a implementarlo. Su sintaxis es:

```
public abstract tipo metodo();
```

Características:

- No tiene cuerpo.
- Se define dentro de una clase abstracta.
- Debe ser implementado por las subclasses concretas.

Permiten representar operaciones que dependen del tipo específico del objeto.

9.2. Jerarquía de clases abstractas

Las clases abstractas pueden formar jerarquías de herencia. Una clase abstracta puede:

- Heredar de otra clase.
- Ser superclase de otras clases abstractas o concretas.

Esto permite modelar problemas donde varias clases comparten estructura y parte del comportamiento, pero difieren en operaciones específicas. Las subclases concretas deben implementar todos los métodos abstractos heredados.

9.3. Interfaces

Una interfaz define únicamente el comportamiento que una clase debe implementar. Se declara con:

`interface`

Una interfaz puede contener:

- Constantes.
- Firmas de métodos públicos.

No contiene implementación de métodos. Las interfaces describen qué puede hacer un objeto, sin indicar cómo lo hará.

9.3.1. Implementación de interfaces

Una clase implementa una interfaz usando:

`implements`

Al hacerlo, debe implementar todos los métodos definidos en la interfaz. Si no implementa alguno, el compilador marca error. Esto asegura que todas las clases que implementan la interfaz compartan el mismo comportamiento externo.

9.4. Referencias a interfaces

Es posible declarar referencias de tipo interfaz. Ejemplo conceptual:

- La referencia es del tipo interfaz.
- El objeto pertenece a una clase que implementa esa interfaz.

No se pueden crear objetos directamente de una interfaz, pero sí referencias. Esto permite aplicar polimorfismo entre objetos de clases distintas que implementan la misma interfaz.

9.5. Implementación de más de una interfaz

Una clase puede implementar varias interfaces simultáneamente. Sintaxis:

```
class Nombre implements I1, I2
```

Esto permite asociar distintos comportamientos a una sola clase. Las interfaces pueden relacionar clases sin necesidad de pertenecer a la misma jerarquía de herencia.

9.6. Definición de grupos de constantes

Como todos los atributos de una interfaz son constantes públicas, una interfaz puede utilizarse para agrupar constantes relacionadas. Esto facilita:

- Organizar valores constantes.
- Compartir constantes entre clases.
- Evitar duplicación.

Las constantes son automáticamente:

- `public`
- `static`
- `final`

9.7. Interfaces definidas en Java

Java proporciona interfaces ya implementadas en su biblioteca estándar. Ejemplo importante:

- `Comparator`

Estas interfaces permiten establecer comportamientos genéricos y reutilizables. El programador implementa la interfaz para adaptar el comportamiento a sus propios objetos.

9.8. Clases abstractas vs. interfaces

Aunque parecen similares, existen diferencias importantes.

- Una clase abstracta puede tener implementación; una interfaz no.
- Una clase abstracta puede tener atributos privados; una interfaz no.
- Una clase abstracta puede tener constructores; una interfaz no.
- Una clase abstracta puede extender otra clase; una interfaz no.

- Una clase puede implementar varias interfaces.
- Una clase solo puede extender una superclase.

En general:

- Clase abstracta: modela una relación de herencia.
- Interfaz: modela comportamiento común.

10. Serializacion de objetos

La serialización es el proceso mediante el cual un objeto se convierte en una secuencia de bytes para poder almacenarse o transmitirse.

Este mecanismo permite guardar el estado de un objeto en un archivo o enviarlo a través de una red.

Posteriormente, esa información puede recuperarse y reconstruir nuevamente el objeto original. La serialización es útil para:

- almacenamiento permanente,
- transferencia de datos,
- persistencia de información.

10.1. Persistencia de objetos

La persistencia consiste en conservar la información de un objeto después de que termina la ejecución del programa.

Normalmente los objetos existen solo en memoria mientras el programa se ejecuta.

La serialización permite almacenar esos objetos en medios externos como archivos. Esto facilita:

- guardar datos,
- recuperar información,
- reutilizar estados previos.

10.2. Interfaz Serializable

Para que un objeto pueda serializarse, su clase debe implementar la interfaz:

`Serializable`

Esta interfaz pertenece al paquete:

`java.io`

No define métodos; simplemente indica que los objetos de esa clase pueden convertirse en bytes.

Ejemplo conceptual:

```
class Persona implements Serializable
```

10.3. Escritura de objetos

La escritura consiste en almacenar el objeto serializado en un archivo. Para ello se utiliza principalmente:

- `FileOutputStream`
- `ObjectOutputStream`

10.3.1. Flujo de salida

Un flujo de salida permite escribir datos en un archivo. `FileOutputStream` establece la conexión con el archivo físico.

10.3.2. Escritura de objetos serializados

`ObjectOutputStream` permite escribir directamente objetos completos. Mediante este flujo se guarda el estado del objeto en el archivo. El método principal es:

```
writeObject()
```

10.4. Lectura de objetos

La lectura permite recuperar objetos almacenados previamente. Se utilizan:

- `FileInputStream`
- `ObjectInputStream`

10.4.1. Flujo de entrada

Un flujo de entrada permite leer datos desde un archivo. `FileInputStream` establece el acceso al archivo almacenado.

10.4.2. Lectura de objetos serializados

`ObjectInputStream` permite reconstruir objetos desde la información almacenada. El método principal es:

`readObject()`

Este método devuelve una referencia al objeto leído.

10.5. Conversion de objetos

Durante la serialización:

- el objeto se convierte en bytes,
- los bytes se almacenan,
- posteriormente se reconstruye.

Este proceso conserva:

- valores de atributos,
- estructura del objeto.

10.6. Objetos no serializables

No todos los objetos pueden serializarse.

Una clase debe implementar `Serializable`; de lo contrario se produce una excepción. También puede haber atributos que no deban almacenarse.

10.6.1. Atributos transient

La palabra reservada:

`transient`

indica que un atributo no será serializado.

Se utiliza cuando ciertos datos no deben persistirse.

Ejemplo:

- contraseñas,
- datos temporales,
- información calculada.

10.7. Version de clases

Cuando una clase cambia entre distintas versiones, la serialización puede generar incompatibilidades. Para controlar esto se utiliza:

`serialVersionUID`

Este identificador permite verificar compatibilidad entre versiones de la clase.

10.8. Excepciones en serializacion

Durante la serialización pueden ocurrir errores. Las más comunes son:

- archivo inexistente,
- error de lectura,
- clase incompatible,
- objeto no serializable.

Por ello normalmente se utiliza manejo de excepciones con:

`try-catch`

10.9. Ventajas de la serializacion

La serialización ofrece:

- almacenamiento de objetos completos,
- recuperación sencilla,
- persistencia automática,
- reutilización de información,
- intercambio de objetos.

Es una herramienta fundamental para manejar archivos y persistencia en aplicaciones orientadas a objetos.

A. Arreglos

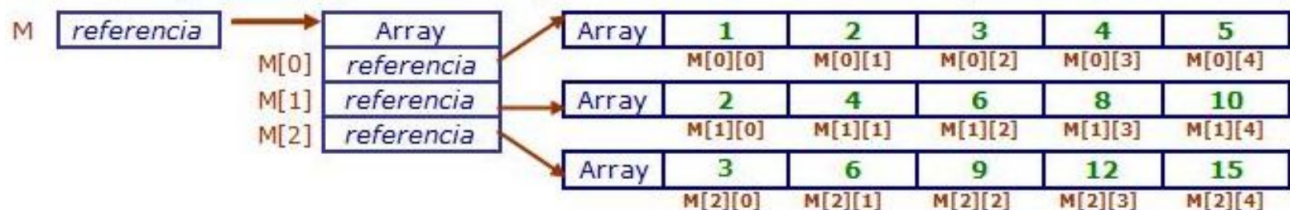
Un arreglo en Java puede tener más de una dimensión. El caso más general son los arreglos bidimensionales también llamados **matrices** o **tablas**.

La dimensión de un arreglo la determina el número de índices necesarios para acceder a sus elementos. Los arreglos que hemos visto han sido unidimensionales porque solo utilizan un índice para acceder a cada elemento. Una matriz necesita dos índices para acceder a sus elementos. Gráficamente podemos representar una matriz como una tabla de n filas y m columnas cuyos elementos son todos del mismo tipo.

La siguiente figura representa un arreglo M de 3 filas y 5 columnas:

	0	1	2	3	4
0	1	2	3	4	5
1	2	4	6	8	10
2	3	6	9	12	15

Pero en realidad una matriz en Java es un arreglo de arreglos. Gráficamente podemos representar la disposición real en memoria del arreglo anterior así:



La longitud del array M (`M.length`) es 3.

La longitud de cada fila del array (`M[i].length`) es 5.

Para acceder a cada elemento de la matriz se utilizan dos índices. El primero indica la fila y el segundo la columna.

Se crean de forma similar a los arreglos unidimensionales, añadiendo un índice. Por ejemplo, una matriz de datos de tipo `int` llamado `ventas` de 4 filas y 6 columnas:

```
int [][] nombreDeArreglo = new int[4][6];
```

Para recorrer una matriz se anidan dos instrucciones de iteración. En general para recorrer un arreglo multidimensional se anidan tantas instrucciones de iteración como dimensiones tenga el arreglo.

Por ejemplo:

```
// Mostramos en pantalla los valores que contiene la matriz
System.out.println("Valores introducidos:");

for (int i = 0; i < A.length; i++) {
    for (int j = 0; j < A[i].length; j++) {
        System.out.print(A[i][j]);
    }
}
```